6 Struct

Questa dispensa propone esercizi sulla scrittura di algoritmi, in linguaggio C, utili alla comprensione della definizione di strutture e di tipi, oltre ad un riepilogo sulla codifica dei numeri con complemento a due.

I tipi di dato strutturato sono dichiarabili in C tramite la parola chiave `struct`

```
void main(){

struct {
        tipo_campo1 nome_campo1;
        tipo_campo2 nome_campo2;
        ...
} nome_struttura;

istruzioni;

}
```

A questo punto potremo utilizzare `nome_struttura` come se fosse una variabile. Per accedere ai campi della struttura si utilizza la sintassi `nome_struttura.nome_campo`. Grazie ad esso possiamo accedere (leggere o scrivere) nei campi della struttura. L'assegnamento tra strutture è possibile solo se le strutture sono dello stesso tipo (stessi campi, per nome e tipo, con stesso ordine di dichiarazione). Non è possibile fare confronti tra strutture in maniera automatica.

Oltre ad i tipi nativi del C (ad esempio `int`, `char` o `float`) è possibile dichiarare dei tipi definiti dall'utente tramite la parola chiave `typedef`. Ad esempio, volendo dichiarare dei vettori della stessa dimensione `n_elementi`:

```
#define LEN n_elementi

typedef tipo_variabile Nuovo_nome_tipo[LEN];

void main(){

Nuovo_nome_tipo nome_variabile;

istruzioni;
}
```

È buona norma dichiarare nuovi tipi con nomi che inizino con una **lettera maiuscola**, con un prefisso o con un suffisso scelto.

La tecnica della dichiarazione di tipo si può combinare con la dichiarazione di strutture. Ad esempio:

```
#define LEN n_elementi

typedef tipo_variabile Nome_tipo[LEN][LEN];

typedef struct {
        tipo_campo1 nome_campo1;
        tipo_campo2 nome_campo2;
        ...
} Nome_tipo_struttura;

void main(){

Nome_tipo nome_variabile;
Nome_tipo_struttura nome_struttura;

istruzioni;
}
```

La definizione di un tipo non implica l'istanziazione di una variabile di quel tipo. La dichiarazione della variabile avverrà di seguito.

Enum Gli enumerated types contengono un elenco di costanti che possono essere indirizzate con valori integer. Per dichiarare tali tipi si utilizza `enum`; vengono dichiarati i tipi e le variabili come nell'esempio che segue:

```
        enum colori {rosso, giallo, verde, blu} pennarello;
        enum giorni {lun,mar,mer,gio,ven,sab,dom} settimana;
        enum colori pulsante, nastro;
```

In tale esempio viene dichiarato `colori` come enumerated type e la variabile `pennarello` con 4 valori accettabili definiti, mentre la variabile `settimana` di tipo `giorni` ha 7 valori accettabili definiti. Le variabili `pulsante` e `nastro` sono di tipo `colori`. Ogni item nell'elenco di valori accettabili è detto enumeration constant. Il C mappa ogni enumeration constant ad un'intero, per cui è ad esempio possibile scrivere:

```
        settimana = verde;
```

che come risultato fa si che settimana abbia valore 2, perchè di default a ogni membro dell'elenco di variabili è assegnato un valore incrementale partendo da 0 per il primo valore. È comunque possibile definire valori diversi agli elementi:

```
enum colori {rosso=10, giallo=30, verde, blu=giallo};
```

Un ulteriore esempio relativo all'assegnazione di valori diversi è il seguente:

```
enum escapes {bell='\a', backspace='\b', tab='\t', newline
   ='\n', vtab='\v', return='\r'};
```

È anche possibile iniziare ad assegnare i valori da un valore iniziale diverso da 0:

```
enum months (jan=1,feb,mar,...dec);
```

dove è implicito che febbraio sia corrispondente al valore 2, marzo al 3 e così via.

6.1 Esercizi

Esercizio 6.1

Un campionato di calcio è composto da venti squadre. Ciascuna squadra ha un nome (lungo al piú 30 caratteri), un numero di punti accumulati durante la stagione e al più 23 calciatori. Ciascun calciatore ha un cognome (lungo al piú 20 caratteri), un'etá, un numero di maglia, un numero di presenze in stagione ed un numero di gol realizzati in stagione.

1. Specificare i tipi Campionato, Squadra e Calciatore.
2. Viene data una variabile di nome serie_a di tipo Campionato, giá inizializzata e riempita dall'utente con tutte le squadre del campionato. Scrivere un frammento di codice che stampi il cognome, il numero di gol realizzati e la squadra di appartenenza del calciatore che ha realizzato piú gol nel campionato.

Esercizio 6.2

Scrivere un programma che permetta all'utente di gestire un libretto di voti di esami, per un massimo di 20 esami. Ogni esame sostenuto ha i seguenti attributi:

- nome del corso (stringa di massimo 256 caratteri)
- voto (minimo 18, massimo 30)
- data (in formato gg/mm/aaaa)
- codice del corso (codice di massimo 6 cifre)

Il programma permette all'utente di svolgere le seguenti operazioni:

- **inserimento:** inserisce un esame in fondo alla lista degli esami eventualmente presenti.
- **stampa:** stampa tutti gli esami presenti, con i dettagli di cui sopra.
- **ricerca:** chiede all'utente di inserire un codice e cerca nel libretto la presenza di un esame corrispondente a quel codice. Se presente, stampa tale esame.
- **uscita:** esce dal programma.

tramite un menù di scelta di questo tipo:

```
[i] inserimento nuovo esame
[s] stampa del libretto
[r] ricerca per codice
[x] uscita
```

Il programma deve continuare a presentare tale menù, fino a quando l'utente non preme il tasto per uscire.

Esercizio 6.3

Si vuole realizzare un programma per la gestione di un magazzino. A tal scopo definiamo un'opportuna struttura `prodotto` definita da:

- un campo `id` di tipo intero che rappresenta il codice univoco del prodotto;
- un campo `nome` di tipo stringa (array di caratteri) di lunghezza massima 50 che rappresenta il nome del prodotto;
- un campo `quantita` di tipo float che rappresenta la quantità disponibile del prodotto;
- un campo `prezzo` di tipo float che rappresenta il prezzo del prodotto.

Si scriva un programma che, dichiarato uno array, di massimo 10 elementi di tipo `prodotto` (il magazzino), permetta di:

- inserire un nuovo prodotto nel magazzino (chiedendo all'utente di inserire i valori dei campi);
- incremetare la quantità di un prodotto esistente (dato il suo `id`);
- visualizzare tutti i prodotti presenti nel magazzino con i loro dettagli;

- vendere un prodotto (dato il suo `id` e la quantità da vendere), aggiornando la quantità disponibile;
- uscire dal programma.

Esercizio 6.4

Le seguenti dichiarazioni definiscono tipi di dati relativi alla categoria degli impiegati di un'azienda (gli impiegati possono essere di prima, seconda, ..., quinta categoria), agli uffici occupati da tali impiegati, all'edificio che ospita tali uffici (edificio diviso in 20 piani ognuno con 40 uffici).

```
/* definizioni dei tipi */
typedef struct {
  char nome[20], cognome[20];
  int cat; // contiene valori tra 1 e 5
  int stipendio;
} Impiegato;

typedef enum{
  nord, nordEst, est, sudEst, sud, sudOvest, ovest, nordOvest
} Esposizione;

typedef struct {
  int superficie; /* in m^2 */
  Esposizione esp;
  Impiegato occupante;
} Ufficio;

/* definizioni delle variabili */
Ufficio torre[20][40];

/* rappresenta un edificio di 20 piani con 40 uffici per piano */
```

1. Si scriva un frammento di codice (che includa eventualmente anche le dichiarazioni di ulteriori variabili) che stampi il cognome, lo stipendio e la categoria di tutte e sole le persone che occupano un ufficio orientato a sud oppure a sudEst e avente una superficie compresa tra 20 e 30 metri quadri;
2. Visualizzi a schermo i piani che non hanno neanche un ufficio esposto a nord;
3. Stampi lo stipendio medio degli impiegati in questi piani che si chiamano "Giacomo" di nome.

Esercizio 6.5

1. Si definiscano le seguenti variabili che specificano le strutture dati per rappresentare i messaggi scambiati attraverso Facebook e ai profili degli utenti:

   - una struttura che definisca un'utente, con nome, cognome e e-mail, tutti composti da caratteri alfanumerici;

   - una struttura che definisca un messaggio, con un contenuto alfanumerico, un mittente scelto tra gli utenti, un numero fissato di destinatari e i destinatari, anch'essi scelti tra gli utenti (fino ad un massimo di 256).

2. Definire, usando il linguaggio C, un'appropriata variabile per memorizzare le informazioni relative a 10 utenti e 10 messaggi. Queste variabili dovranno essere chiamate `utenti_facebook` e `messaggi_inviati`.

3. Scrivere in linguaggio C, aggiungendo eventualmente opportune dichiarazioni di variabili, un frammento di codice che permetta di visualizzare a video l'indirizzo email di tutti gli utenti che hanno spedito almeno un messaggio e che non siano tra i destinatari di tale messaggio. Si presupponga che la variabile `messaggi_inviati` sia inizializzata correttamente con le informazioni relative a 10 messaggi. Si presupponga inoltre che l'indirizzo email sia identificatore univoco di un utente: pertanto si può utilizzare l'indirizzo email per verificare se due utenti sono lo stesso utente.

Soluzioni

Soluzione dell'esercizio 6.1

1.
```
# define MAX_SQUADRE 20
# define MAX_COGNOME 20
# define MAX_NOME 30
# define MAX_CALCIATORI 23

typedef struct{
    char cognome[MAX_COGNOME + 1];
    int eta;
    int maglia;
    int presenze;
    int gol;
}Calciatore;

typedef struct{
    char nome[MAX_NOME + 1];
    int punti;
    Calciatore calciatore[MAX_CALCIATORI];
}Squadra;

typedef Squadra Campionato[MAX_SQUADRE];
```

2.
```
Campionato serie_a;
Calciatore capocannoniere;
Squadra squadra_capocannoniere;
int i, j;

// inizializzazione capocannoniere
capocannoniere = serie_a[0].calciatore[0];
squadra_capocannoniere = serie_a[0];

// ricerca capocannoniere
for(i = 0; i < MAX_SQUADRE; i++)
    for(j = 0; j < MAX_CALCIATORI; j++)
        if(serie_a[i].calciatore[j].gol > capocannoniere.gol){
            capocannoniere = serie_a[i].calciatore[j];
            squadra_capocannoniere = serie_a[i];
        }

// stampa capocannoniere
printf("\ncapocannoniere: %s\n", capocannoniere.cognome);
printf("squadra: %s\n", squadra_capocannoniere.nome);
printf("numero di gol: %d\n\n", capocannoniere.gol);
```

Soluzione dell'esercizio 6.2

```
#include <stdio.h>

#define DIM_LIBR 20
#define DIM_NOME 256
#define DIM_DATA 10
#define DIM_CODI 6

typedef struct{
```

```
    char nome_corso[DIM_NOME+1];
    int voto;
    char data[DIM_DATA+1];
    char codice[DIM_CODI+1];
} Esame;

int main(){

    //dichiarazioni
    char menu;
    int n_esami = 0;
    int i,j;
    char codice[DIM_CODI];
    int is_equal;
    char cestino;
    //dichiarazione con tipo user-defined
    Esame libretto[DIM_LIBR];

    do {
        system("cls"); //WINDOWS
        //system("clear"); //UNIX
        printf("Scegliere una delle seguenti opzioni:\n");
        printf("[i] inserimento nuovo esame\n");
        printf("[s] stampa del libretto\n");
        printf("[r] ricerca per codice\n");
        printf("[x] uscita\n");
        scanf("%c", &menu);
        scanf("%c", &cestino);

        switch (menu) {
            case 'i':
                //Istruzioni
                printf("Inserisci nome esame: ");
                scanf("%s",libretto[n_esami].nome_corso);
                printf("Inserisci voto: ");
                scanf("%d",&libretto[n_esami].voto);
                printf("Inserisci data: ");
                scanf("%s",libretto[n_esami].data);
                printf("Inserisci codice: ");
                scanf("%s",libretto[n_esami].codice);
                n_esami++;

                break;
            case 's':
                //Istruzioni
                for (i = 0; i < n_esami; i++)
                   // printf() ....

                break;
            case 'r':
                //Istruzioni
                printf("Inserisci il codice da controllare: ");
                scanf("%s",codice);
                for (i = 0; i < n_esami; i++) {
                    //libretto[i].codice == codice;
                    is_equal = 1;
                    for (j = 0; j < DIM_CODI; j++)
                        if (libretto[i].codice[j] != codice[j])
                            is_equal = 0;

                    if (is_equal){
                        //Stampare esame
```

```
                    }
                }

                break;

            default:

                break;
        }
    } while (menu != 'x');

}
```

Soluzione dell'esercizio 6.3

```
#include <stdio.h>
#include <string.h>

#define MAX_PRODOTTI 10
#define MAX_NOME 50

/* Definizione struttura prodotto */
typedef struct {
    int id;
    char nome[MAX_NOME + 1];
    float quantita;
    float prezzo;
} Prodotto;

int main() {
    Prodotto magazzino[MAX_PRODOTTI];
    int n_prodotti = 0;
    int scelta;
    int i, id, trovato;
    float qta;

    do {
        printf("\n--- MENU MAGAZZINO ---\n");
        printf("1. Inserisci nuovo prodotto\n");
        printf("2. Incrementa quantita di un prodotto\n");
        printf("3. Visualizza tutti i prodotti\n");
        printf("4. Vendi un prodotto\n");
        printf("5. Esci\n");
        printf("Scelta: ");
        scanf("%d", &scelta);

        switch (scelta) {
            case 1:
                if (n_prodotti >= MAX_PRODOTTI) {
                    printf("Magazzino pieno!\n");
                } else {
                    printf("Inserisci id: ");
                    scanf("%d", &id);

                    /* controllo ID duplicato */
                    trovato = 0;
                    for (i = 0; i < n_prodotti; i++) {
                        if (magazzino[i].id == id) {
                            trovato = 1;
                            break;
```

```
                }
            }

            if (trovato) {
                printf("Errore: esiste gia' un prodotto con questo ID.\n")
                    ;
            } else {
                magazzino[n_prodotti].id = id;
                printf("Inserisci nome: ");
                scanf("%50s", magazzino[n_prodotti].nome);
                printf("Inserisci quantita: ");
                scanf("%f", &magazzino[n_prodotti].quantita);
                printf("Inserisci prezzo: ");
                scanf("%f", &magazzino[n_prodotti].prezzo);
                n_prodotti++;
                printf("Prodotto inserito correttamente.\n");
            }
        }
        break;

    case 2:
        printf("Inserisci id del prodotto da incrementare: ");
        scanf("%d", &id);
        trovato = 0;
        for (i = 0; i < n_prodotti; i++) {
            if (magazzino[i].id == id) {
                printf("Inserisci quantita da aggiungere: ");
                scanf("%f", &qta);
                if (qta <= 0) {
                    printf("Quantita non valida.\n");
                } else {
                    magazzino[i].quantita += qta;
                    printf("Quantita aggiornata: %.2f\n", magazzino[i].
                        quantita);
                }
                trovato = 1;
                break;
            }
        }
        if (!trovato)
            printf("Prodotto non trovato.\n");
        break;

    case 3:
        if (n_prodotti == 0) {
            printf("Nessun prodotto nel magazzino.\n");
        } else {
            printf("\n--- Elenco prodotti ---\n");
            for (i = 0; i < n_prodotti; i++) {
                printf("ID: %d | Nome: %s | Quantita: %.2f | Prezzo: %.2f\
                    n",
                       magazzino[i].id,
                       magazzino[i].nome,
                       magazzino[i].quantita,
                       magazzino[i].prezzo);
            }
        }
        break;

    case 4:
        printf("Inserisci id del prodotto da vendere: ");
        scanf("%d", &id);
```

```
                trovato = 0;
                for (i = 0; i < n_prodotti; i++) {
                    if (magazzino[i].id == id) {
                        printf("Inserisci quantita da vendere: ");
                        scanf("%f", &qta);
                        if (qta <= 0) {
                            printf("Quantita non valida.\n");
                        } else if (qta > magazzino[i].quantita) {
                            printf("Errore: quantita richiesta superiore alla
                                disponibilita' (%.2f)\n",
                                    magazzino[i].quantita);
                        } else {
                            magazzino[i].quantita -= qta;
                            printf("Vendita effettuata. Nuova quantita: %.2f\n",
                                magazzino[i].quantita);
                        }
                        trovato = 1;
                        break;
                    }
                }
                if (!trovato)
                    printf("Prodotto non trovato.\n");
                break;

            case 5:
                printf("Uscita dal programma.\n");
                break;

            default:
                printf("Scelta non valida.\n");
                break;
        }

    } while (scelta != 5);

    return 0;
}
```

Soluzione dell'esercizio 6.4

```
int p, u; /* indice di piano nelledificio e di ufficio nel piano */

for (p = 0; p < 20; p++)
    for (u = 0; u < 40; u++)
        if ((torre[p][u].esp == sudEst || torre[p][u].esp == sud) &&
            (torre[p][u].superficie >=20 && torre[p][u].superficie<=30)) {
            printf("\n il Signor %s è impiegato di categoria %d",
                   torre[p][u].occupante.cognome,
                   torre[p][u].occupante.cat);
            printf("e ha uno stipendio pari a %d euro \n",
              torre[p][u].occupante.stipendio);
        }
```

```
int uffNord; /* uffNord fa da flag*/

for (p = 0; p < 20; p++) {
    uffNord = 0; //per ogni piano assumo 0
    for (u = 0; u < 40 && !uffNord; u++)
        if(torre[p][u].esposizione == nord)
            uffNord = 1;
```

```
    /* se qui vale ancora 0 vuol dire che non ci sono uffici a nord*/
    if (!uffNord)
        printf("il piano %d  non ha edifici esposti a nord", p);
 }

/* è corretto anche senza !uffNord nel for(), anche se non è efficiente. */
```

```
for(i = 0; i < 20; i++) { //scorre i piani
    noUfficiNord = 1; //versione con flag inversa

    for(j = 0; j < 40; j++) //senza flag arriva fino a 39
        if(torre[i][j].esposizione == nord)
            noUfficiNord = 0;

    if(noUfficiNord) {
        printf("il piano %d non ha uffici a nord", i);
        stipendioMedio = 0;
        cnt = 0;
        for(j = 0; j < 40; j++)
            if(strcmp(torre[i][j].occupante.nome, "Giacomo") == 0){
                stipendioMedio += torre[i][j].occupante.stipendio;
                cnt++;
            }
        printf("Lo stipendio medio dei Giacomo nel "
               "piano è %f ", stipendioMedio / cnt);
    }
 }
```

Soluzione dell'esercizio 6.5

```
typedef struct {
  char nome[24];
  char cognome[24];
  char email[64];
} Utente;

typedef struct {
  char contenuto[256];
  Utente mittente;
  int numDestinatari;
  Utente destinatari[256];
} Messaggio;
```

```
Utente utenti_facebook[10];
Messaggio messaggi_inviati[10];
```

Gli utenti che hanno spedito almeno un messaggio sono necessariamente presenti come mittenti nella variabile `messaggi_inviati`:

```
void main () {
  int i, j;
  Utente utenti_facebook[10];
  Messaggio messaggi_inviati[10];

  for (i = 0; i < 10; i++) {
    j = 0;
```

```
    //ricerca destinatario.email != mittente.email con strcmp()
    while (j < messaggi_inviati[i].numDestinatari &&
              strcmp(messaggi_inviati[i].mittente.email,
                     messaggi_inviati[i].destinatari[j].email) != 0) {
      j++;
    }

    if (j == messaggi_inviati[i].numDestinatari) //non trovato
      printf("%s", Messaggi[i].mittente.email);
}
```